OTEL-08: Troubleshooting OTel Pipelines

> **Series:** OTEL — OpenTelemetry Integration | **Notebook:** 8 of 8 | **Created:** January 2026 | **Last Updated:** 10/28/2026

Debugging OpenTelemetry Data Flows

When telemetry data doesn't arrive as expected, systematic troubleshooting is essential. This notebook covers common issues, diagnostic techniques, and resolution steps for OpenTelemetry pipelines.

Table of Contents

1. Troubleshooting Methodology
2. Collector Issues
3. SDK Issues
4. Connectivity Issues
5. Data Quality Issues
6. Performance Issues
7. Diagnostic Tools
8. Common Fixes

Prerequisites

Requirement	Details
Knowledge	OTEL-01 through OTEL-07
Access	Collector logs, application logs

1. Troubleshooting Methodology

Systematic Approach

```
1. Identify the symptom
   ↓
2. Check component by component
   SDK → Collector → Backend
   ↓
3. Verify configuration
   ↓
4. Test connectivity
   ↓
5. Apply fix
   ↓
6. Verify resolution
```

Symptom Categories

Symptom	Likely Issue	Start Here
No data at all	Connectivity, auth	Section 4
Partial data	Filtering, sampling	Section 5
Delayed data	Performance	Section 6
Wrong data	Configuration	Section 5
Collector crashes	Resources	Section 6

2. Collector Issues

Collector Won't Start

Check configuration syntax:

```
# Validate config
otelcol-contrib validate --config=config.yaml
```

Common configuration errors:

Error	Cause	Fix
`unknown type`	Invalid component name	Check spelling, verify contrib has component
`duplicate key`	YAML syntax error	Check indentation
`no pipelines defined`	Missing service section	Add service.pipelines

Enable Debug Logging

```
service:
  telemetry:
    logs:
      level: debug
```

Collector Dropping Data

Check queue status:

```
# If using zpages extension
curl http://localhost:55679/debug/tracez
```

Common causes:

Cause	Indicator	Fix
Memory pressure	OOM, restarts	Increase memory_limiter limits

Cause	Indicator	Fix
Export failure	Retry logs	Check backend connectivity
Queue full	Queue overflow logs	Increase queue size or consumers

3. SDK Issues

No Spans Generated

Python:

```
# Enable SDK debug logging
import logging
logging.basicConfig(level=logging.DEBUG)

# Verify tracer provider is set
from opentelemetry import trace
print(trace.get_tracer_provider())
# Should NOT be NoOpTracerProvider
```

Java:

```
# Enable agent debug
java -javaagent:opentelemetry-javaagent.jar \
  -Dotel.javaagent.debug=true \
  -jar app.jar
```

Common SDK Issues

Issue	Cause	Fix
NoOpTracer	Provider not set	Call set_tracer_provider()
No export	Processor not added	Add BatchSpanProcessor
Missing spans	Not in context	Use start_as_current_span
Lost context	Async not handled	Use context propagation

Async Context in Python

OpenTelemetry Python keeps the current span in a `ContextVar`, and `asyncio` copies the current context into every task it creates. A span started with `start_as_current_span` is therefore already current inside coroutines run with `await`, `asyncio.gather` or `asyncio.create_task` — wrapping `gather` in `context.attach(context.get_current())` changes nothing. If spans are missing parents in async code, look for work that does not run as a task created under the span.

> **Sources:** asyncio — Coroutines and Tasks (Python docs) — `create_task`: *"The current context copy is created when no context is provided."*; `gather`: *"If any awaitable in aws is a coroutine, it is automatically scheduled as a Task."*; contextvars_context.py (OpenTelemetry GitHub) — *"An implementation of the RuntimeContext interface which wraps ContextVar under the hood."*

4. Connectivity Issues

Test Collector Connectivity

```
# Test OTLP endpoint
curl -v http://collector:4318/v1/traces \
  -X POST \
  -H "Content-Type: application/json" \
  -d '{}'

# Test gRPC (with grpcurl)
grpcurl -plaintext collector:4317 list
```

Test Dynatrace Connectivity

```
# Test OTLP endpoint (set DT_API_TOKEN env var first).
# Dynatrace accepts binary protobuf only ("JSON is not supported for Protocol
Buffers"),
# so send an empty protobuf body — a valid, empty export request.
curl -v https://${DT_ENV_ID}.live.dynatrace.com/api/v2/otlp/v1/traces \
  -X POST \
  -H "Authorization: Api-Token ${DT_API_TOKEN}" \
  -H "Content-Type: application/x-protobuf" \
  --data-binary ''

# Expected: a 2xx status. 401 = token or auth scheme (Api-Token vs Bearer);
403 = missing scope.
```

Common Connectivity Errors

Error	Cause	Fix
`connection refused`	Wrong port/host	Verify endpoint
`401 Unauthorized`	Invalid token, or wrong scheme (`Api-Token` for classic tokens, `Bearer` for platform tokens)	Check token and scheme
`403 Forbidden`	Missing scope	Add required scope to token
`certificate error`	TLS issue	Check certs, disable verify for test
`timeout`	Network/firewall	Check firewall rules

Proxy Issues

```
# Collector proxy configuration
exporters:
  otlphttp:
    endpoint: https://tenant.live.dynatrace.com/api/v2/otlp
    proxy_url: http://proxy.example.com:8080
```

```
# Or via environment
export HTTPS_PROXY=http://proxy.example.com:8080
```

5. Data Quality Issues

Missing Attributes

Debug exporter to see data:

```
exporters:
  debug:
    verbosity: detailed

service:
  pipelines:
    traces:
      exporters: [debug, otlphttp]
```

service.name Missing

Without `service.name`, OpenTelemetry SDKs report the service as `unknown_service` (plus the process name, in some SDKs), so its spans land under a meaningless service. Set it in the SDK or, as a fallback, in the Collector:

```
# Add default service.name in collector
processors:
  resource:
    attributes:
      - key: service.name
        value: "unknown-service"
        action: insert  # Only adds if missing
```

Wrong Timestamps in File Logs

OTLP timestamps are Unix-epoch nanoseconds and carry no time zone, so the Collector's own `TZ` does not affect OTLP data. Time-zone errors appear when a receiver **parses** a timestamp written without a zone, for example a `file_log` time parser reading local times. Set the parser's location there, rather than changing the container's `TZ`.

Data Being Filtered

Check filter processor configuration:

```
processors:
  filter:
    traces:
      span:
        # This filters OUT matching spans
        - 'attributes["url.path"] == "/health"'
```

6. Performance Issues

Collector Memory Issues

Symptoms:
- OOMKilled restarts
- Slow processing
- Data drops

Fixes:

```
processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 1500      # Increase limit
    spike_limit_mib: 500 # Allow spikes

  batch:
    timeout: 5s           # Faster flush
    send_batch_size: 500  # Smaller batches
```

High Latency

Causes and fixes:

Cause	Indicator	Fix
Large batches	Delayed export	Reduce batch size
Slow export	Backend latency	Increase parallelism

Cause	Indicator	Fix
Too many processors	Processing time	Remove unnecessary processors

```
exporters:
  otlphttp:
    sending_queue:
      num_consumers: 10  # More parallel exports
```

SDK Overhead

```
# Use sampling to reduce volume
from opentelemetry.sdk.trace.sampling import TraceIdRatioBased

provider = TracerProvider(
    sampler=TraceIdRatioBased(0.1),  # 10% sampling
    resource=resource
)
```

7. Diagnostic Tools

Collector zpages

```
extensions:
  zpages:
    endpoint: 0.0.0.0:55679

service:
  extensions: [zpages]
```

Access:
- `http://collector:55679/debug/servicez` - Service info
- `http://collector:55679/debug/pipelinez` - Pipeline status
- `http://collector:55679/debug/extensionz` - Extensions
- `http://collector:55679/debug/tracez` - Trace samples

Health Check

```
extensions:
  health_check:
    endpoint: 0.0.0.0:13133
```

```
curl http://collector:13133/
# {"status":"Server available"}
```

pprof Profiling

```
extensions:
  pprof:
    endpoint: 0.0.0.0:1777
```

```
go tool pprof http://collector:1777/debug/pprof/heap
```

```
// Check OTel data arrival (OTLP-ingested spans, 5-minute buckets)
fetch spans, from:-1h
| filter dt.openpipeline.source == "/api/v2/otlp/v1/traces"
| summarize count = count(), by:{time_bucket = bin(start_time, 5m)}
| sort time_bucket asc
```

```
// Find OTLP-ingested spans with no usable service.name.
// Restrict to OTLP: OneAgent spans rarely carry service.name (1,739 of
303,492 in an hour on
// 10/02/2026 — they use dt.service.name), so an unrestricted
isNull(service.name) lists
// nearly every OneAgent span as a false positive.
fetch spans, from:-1h
| filter dt.openpipeline.source == "/api/v2/otlp/v1/traces"
| filter isNull(service.name) or service.name == "" or
startsWith(service.name, "unknown_service")
| summarize count = count(), by:{otel.scope.name}
| sort count desc
```

8. Common Fixes

Quick Reference

Problem	Fix
No data at all	Check endpoint, auth token, network
No service entity	Add service.name resource attribute
Collector OOM	Increase memory_limiter, reduce batch size
High cardinality	Remove dynamic attributes
Context lost	Use context propagation in async
Wrong time (file logs)	Set the location on the file_log time parser, not the container TZ (§ 5)
SSL errors	Check certs, add CA

Collector Config Template

A production-ready template:

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 1500
    spike_limit_mib: 500
  batch:
    timeout: 10s
    send_batch_size: 1000
  resource:
    attributes:
      - key: service.name
        value: unknown
        action: insert

exporters:
  otlphttp:
    endpoint: https://${DT_ENV}.live.dynatrace.com/api/v2/otlp
    headers:
      Authorization: Api-Token ${DT_TOKEN}
    retry_on_failure:
      enabled: true
      initial_interval: 5s
      max_interval: 30s

extensions:
  health_check:
  zpages:

service:
  extensions: [health_check, zpages]
  telemetry:
    logs:
      level: info
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, resource, batch]
      exporters: [otlphttp]
    metrics:
```

```
      receivers: [otlp]
      processors: [memory_limiter, resource, batch]
      exporters: [otlphttp]
    logs:
      receivers: [otlp]
      processors: [memory_limiter, resource, batch]
      exporters: [otlphttp]
```

Summary

In this notebook, you learned:

- Systematic troubleshooting methodology
- Collector debugging and configuration validation
- SDK issue diagnosis and async context handling
- Connectivity testing and common errors
- Data quality issue resolution
- Performance tuning for memory and latency
- Diagnostic tools: zpages, health check, pprof
- Quick fixes for common problems

References

- Collector Troubleshooting
- Python instrumentation (opentelemetry.io)
- Dynatrace OTel Troubleshooting

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.